ELEC3612 — Pattern Recognition and Machine Intelligence

Assignment 1: Machine Learning Model Implementation and Reporting

University of Sydney | Individual Assignment

Martí Casas Niubó

SID: 560458080

1. Introduction

1.1 Dataset Description

This assignment uses the Kaggle dataset "Job Change of Data Scientists", which contains information about people who have completed data science training courses. The dataset has 19,158 instances and 14 features, covering demographic information, education background, work experience, and company details. The target variable is binary, indicating whether a candidate is looking for a job change (1) or not (0).

The dataset presents some realistic challenges. It contains significant missing values in several columns (up to 32% in company_type and company_size), it is class-imbalanced (75% class 0 vs 25% class 1), and it includes a mix of numerical, ordinal, and nominal categorical features. An initial exploration of the data showed some patterns: candidates with less experience tend to show higher job-seeking rates, those from cities with lower development indices are more likely to seek a change, and candidates enrolled in full-time courses show higher job-seeking rates than those not enrolled. The training_hours variable is right-skewed, with most candidates completing under 100 hours. All these patterns suggest that career stage and location context are likely informative features for the classification task.

1.2 Motivation for Model Selection

We implement and evaluate four supervised classifiers representing two model families, one unsupervised model, and four ensemble methods:

- Tree-based models (Decision Tree, Random Forest): straightforward to interpret and effective with mixed feature types.
- Non-tree-based models (Logistic Regression, SVM): use different approaches to find decision boundaries, providing a useful contrast with tree-based methods.
- K-Means clustering: used to explore whether the data has a natural group structure that aligns with the known class labels.
- Ensemble methods (Bagging, Boosting, Voting, Stacking): used to test whether combining multiple models improves performance over individual classifiers.

2. Methods

2.1 Preprocessing Workflow

The preprocessing pipeline was built using scikit-learn's Pipeline and ColumnTransformer, ensuring that all transformations are fitted only on training data to prevent data leakage. The pipeline applies different treatments to each column type:

- Numerical columns (city_development_index, training_hours): median imputation followed by StandardScaler normalisation.
- Ordinal columns (experience, last_new_job): most-frequent imputation followed by OrdinalEncoder.
- Nominal columns (city, gender, relevent_experience, enrolled_university, education_level, major_discipline, company_size, company_type): most-frequent imputation followed by OneHotEncoder.

The enrollee_id column was dropped as it is an identifier with no predictive value. The dataset is class-imbalanced (75%/25%), which was taken into account by using stratified splits throughout all experiments to ensure both classes are proportionally represented in train and test sets. Weighted averaging was also used for all performance metrics for the same reason.

2.2 Model Descriptions

Decision Tree (DT): A tree-based classifier that recursively partitions the feature space by selecting the split that maximises information gain. Hyperparameters tuned: max_depth, min_samples_split, criterion.

Random Forest (RF): An ensemble of decision trees trained on bootstrap samples with random feature subsets. We varied the number of estimators (50, 100, 150, 200) to assess the effect of ensemble size.

Support Vector Machine (SVM): A classifier that finds the maximum-margin hyperplane separating classes. We used an RBF kernel with default hyperparameters (C=1.0, gamma=scale).

Logistic Regression (LR): A linear classifier that models the probability of class membership. Hyperparameters tuned: C (regularisation strength), penalty (l1/l2).

K-Means Clustering: An unsupervised algorithm that partitions data into k clusters by minimising intra-cluster variance. We used k=2 to match the number of known classes, with k-means++ initialisation.

Ensemble Methods: Bagging (BaggingClassifier with DT base), AdaBoost, Gradient Boosting, Voting Classifier (hard and soft), and Stacking (DT + LR + RF with LR meta-learner).

2.3 Evaluation Strategy

All supervised models were evaluated using four primary metrics: accuracy, weighted precision, weighted recall, and weighted F1-score. Weighted averaging was used throughout to account for the class imbalance. Confusion matrices were generated for each model to provide a more detailed view of classification errors, distinguishing between false positives and false negatives for each class.

K-Means was evaluated using Silhouette Score (internal validation), ARI (Adjusted Rand Index, external validation), and AMI (Adjusted Mutual Information, external validation).

For the robustness analysis (Decision Tree and Logistic Regression), we systematically varied:

- Train-test split ratios: 60/40, 70/30, 80/20
- Stratified vs non-stratified splits
- K-fold CV values: k = 3, 5, 10 (StratifiedKFold)
- Random seeds: 42 and 123
- With and without GridSearchCV hyperparameter tuning

3. Results

3.1 Decision Tree — Split Ratio and Seed Analysis

Table 1 shows the effect of varying train-test split ratios and random seeds on Decision Tree performance (no hyperparameter tuning).

Split	Seed	Accuracy	Precision	Recall	F1
60/40	42	0.6986	0.7016	0.6986	0.7001
70/30	42	0.7074	0.7098	0.7074	0.7086
80/20	42	0.7035	0.7079	0.7035	0.7056
60/40	123	0.7011	0.7056	0.7011	0.7033
70/30	123	0.7138	0.7143	0.7138	0.7140
80/20	123	0.7109	0.7121	0.7109	0.7115

Table 1. Decision Tree performance across split ratios and seeds.

3.2 Decision Tree — Stratified vs Non-Stratified Split

Split Type	Class 1 % in Test	Accuracy	Precision	Recall	F1
Stratified	0.2492	0.7035	0.7079	0.7035	0.7056
Non-stratified	0.2484	0.7106	0.7133	0.7106	0.7119

Table 2. Effect of stratified vs non-stratified split on Decision Tree.

3.3 Decision Tree — K-Fold CV Analysis

k	Acc Mean	Acc Std	Prec Mean	Rec Mean	F1 Mean	F1 Std
3	0.7000	0.0065	0.7028	0.7000	0.7013	0.0070
5	0.7019	0.0076	0.7037	0.7019	0.7027	0.0069
10	0.7042	0.0086	0.7084	0.7042	0.7061	0.0076

Table 3. Decision Tree cross-validation results across different k values.

3.4 Decision Tree — Hyperparameter Tuning

Setup	Accuracy	Precision	Recall	F1
No tuning	0.7035	0.7079	0.7035	0.7056
With tuning (GridSearchCV)	0.7863	0.7721	0.7863	0.7758

Table 4. Effect of hyperparameter tuning on Decision Tree. Best params: criterion=entropy, max_depth=3, min_samples_split=2.

Figure 1 shows the confusion matrices for the Decision Tree before and after tuning. Taking class 1 (job-seeking) as the positive class, the tuned model reduces false positives (instances of class 0 incorrectly predicted as class 1) from 589 to 295, and increases true positives in the minority class from 408 to 431.

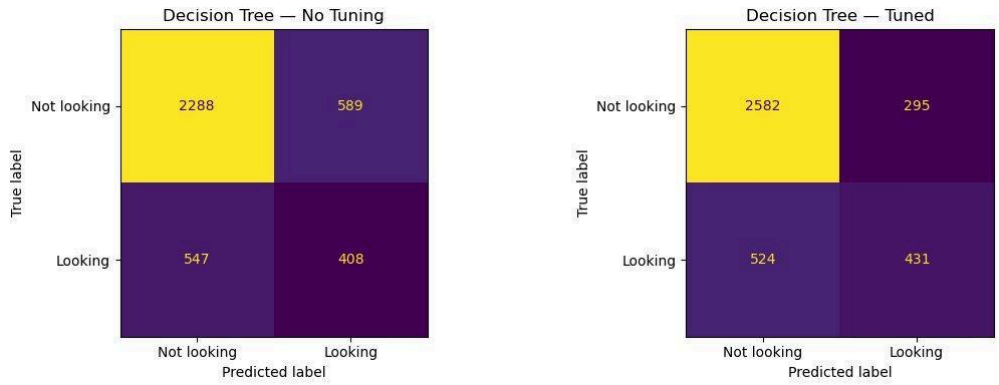

Figure 1. Confusion matrices for Decision Tree: no tuning (left) vs tuned (right).

3.5 Logistic Regression — Split Ratio and Seed Analysis

Split	Seed	Accuracy	Precision	Recall	F1
60/40	42	0.7774	0.7547	0.7774	0.7548
70/30	42	0.7778	0.7547	0.7778	0.7538
80/20	42	0.7777	0.7548	0.7777	0.7547
60/40	123	0.7800	0.7572	0.7800	0.7540
70/30	123	0.7839	0.7625	0.7839	0.7577
80/20	123	0.7831	0.7615	0.7831	0.7583

Table 5. Logistic Regression performance across split ratios and seeds.

3.6 Logistic Regression — Stratified vs Non-Stratified Split

Split Type	Class 1 % in Test	Accuracy	Precision	Recall	F1
Stratified	0.2492	0.7777	0.7548	0.7777	0.7547
Non-stratified	0.2484	0.7758	0.7521	0.7758	0.7523

Table 6. Effect of stratified vs non-stratified split on Logistic Regression.

3.7 Logistic Regression — K-Fold CV Analysis

k	Acc Mean	Acc Std	Prec Mean	Rec Mean	F1 Mean	F1 Std
3	0.7746	0.0019	0.7499	0.7746	0.7485	0.0013
5	0.7747	0.0048	0.7498	0.7747	0.7480	0.0059
10	0.7752	0.0067	0.7505	0.7752	0.7486	0.0088

Table 7. Logistic Regression cross-validation results across different k values.

3.8 Logistic Regression — Hyperparameter Tuning

Setup	Accuracy	Precision	Recall	F1
No tuning	0.7777	0.7548	0.7777	0.7547
With tuning (GridSearchCV)	0.7777	0.7548	0.7777	0.7547

Table 8. Effect of hyperparameter tuning on Logistic Regression. Best params: C=1, penalty=l1, solver=liblinear.

Figure 2 shows the confusion matrices for Logistic Regression. Since tuning produced identical results, both matrices are the same, confirming that the default hyperparameters are already optimal for this dataset.

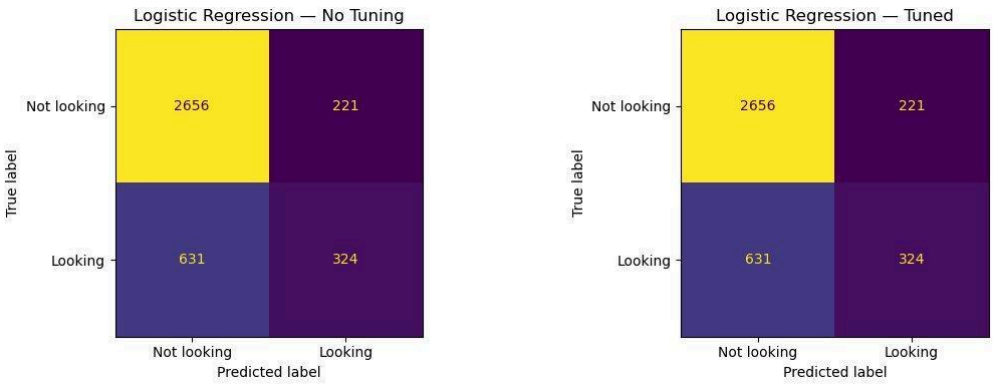

Figure 2. Confusion matrices for Logistic Regression: no tuning (left) vs tuned (right).

3.9 Random Forest — Number of Estimators

n_estimators	Accuracy	Precision	Recall	F1
50	0.7657	0.7485	0.7657	0.7538
100	0.7670	0.7498	0.7670	0.7549
150	0.7662	0.7491	0.7662	0.7543
200	0.7701	0.7541	0.7701	0.7590

Table 9. Random Forest performance with varying number of estimators (80/20 split, seed=42).

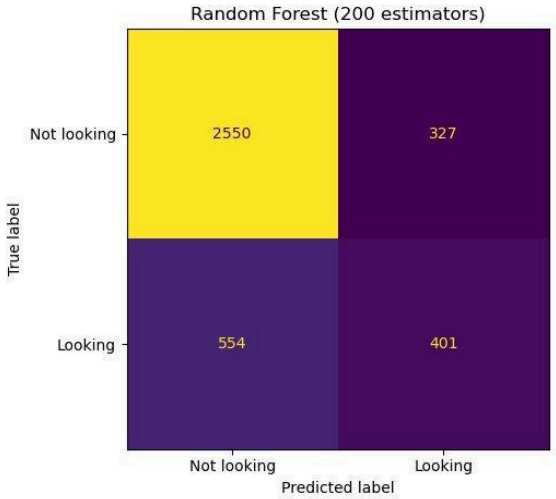

Figure 3. Confusion matrix for Random Forest (200 estimators).

3.10 SVM

Model	Accuracy	Precision	Recall	F1
SVM (RBF, C=1.0)	0.7784	0.7567	0.7784	0.7577

Table 10. SVM performance (80/20 split, seed=42).

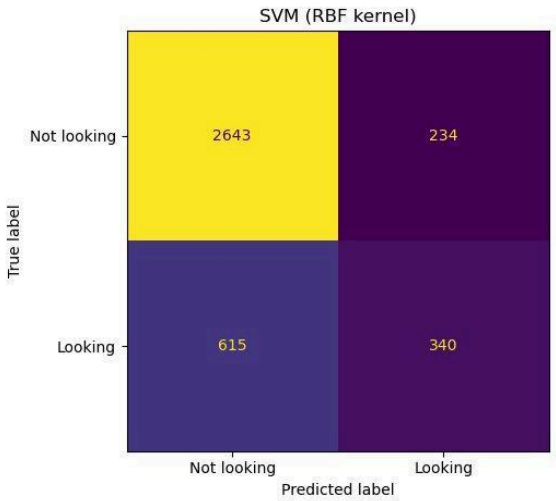

Figure 4. Confusion matrix for SVM (RBF kernel).

3.11 K-Means Clustering

K-Means was applied to the full preprocessed feature matrix (160 features after one-hot encoding) with k=2 clusters and k-means++ initialisation. The elbow method was used to inform the choice of k.

Model	Silhouette Score	ARI	AMI
K-Means (k=2)	0.5277	-0.0180	0.0034

Table 11. K-Means clustering evaluation metrics.

The Silhouette Score of 0.53 indicates moderately compact clusters. However, the near-zero ARI (-0.018) and AMI (0.003) show that the clusters found by K-Means do not correspond meaningfully to the true class labels, suggesting the data does not have a clear two-cluster geometric structure aligned with job-seeking behaviour.

3.12 Ensemble Models

All ensemble models were evaluated on the same 80/20 stratified split (seed=42) as the base models for a fair comparison.

Type	Model	Accuracy	Precision	Recall	F1
Base	Decision Tree (tuned)	0.7863	0.7721	0.7863	0.7758
Ensemble	Gradient Boosting	0.7858	0.7719	0.7858	0.7757
Ensemble	Stacking	0.7813	0.7608	0.7813	0.7619
Base	Random Forest (200 trees)	0.7701	0.7541	0.7701	0.7590
Ensemble	Bagging (DT)	0.7675	0.7538	0.7675	0.7586
Base	SVM (RBF)	0.7784	0.7567	0.7784	0.7577
Ensemble	Voting (hard)	0.7704	0.7521	0.7704	0.7569
Base	Logistic Regression	0.7777	0.7548	0.7777	0.7547
Ensemble	AdaBoost	0.7769	0.7528	0.7769	0.7503
Ensemble	Voting (soft)	0.7518	0.7386	0.7518	0.7438
Base	Decision Tree (no tuning)	0.7035	0.7079	0.7035	0.7056

Table 12. All models ranked by F1-score: ensemble vs base models.

4. Discussion

4.1 Effect of Split Ratio

In these experiments, the 70/30 split tended to produce slightly better results for both Decision Tree and Logistic Regression. The 60/40 split generally performed worse, likely because a smaller training set limits the model's ability to learn patterns. The differences between ratios were generally small, which suggests the dataset is large enough that moderate changes in training size have a limited impact.

4.2 Effect of Stratification

The difference between stratified and non-stratified splits was minimal for both models. The class proportions in the test set were nearly identical (0.2492 vs 0.2484), which is expected given the large dataset size (19,158 instances). With sufficient data, random splits naturally approximate the class distribution. Nevertheless, stratification remains recommended as best practice to guarantee consistent evaluation, particularly with smaller datasets.

4.3 Effect of K-Fold CV

For Decision Tree, increasing k from 3 to 10 improved mean accuracy slightly (0.7000 to 0.7042) but also increased variance in F1 (std from 0.0070 to 0.0076). For Logistic Regression, results were very stable across all k values (F1 mean between 0.7480 and 0.7486), suggesting LR is less sensitive to how the training data is partitioned during cross-validation.

4.4 Effect of Hyperparameter Tuning

Tuning had a key effect on the Decision Tree, improving F1 from 0.7056 to 0.7758. The optimal configuration used shallow trees (max_depth=3) with entropy criterion, which acts as a regulariser preventing overfitting. The confusion matrix in Figure 1 confirms this improvement: taking class 1 (job-seeking) as the positive class, the tuned model reduces false positives from 589 to 295 and increases true positives from 408 to 431, showing clear improvement on both classes. In contrast, Logistic Regression showed no improvement from tuning, as its default hyperparameters were already well-suited to this dataset.

4.5 Comparison Across Supervised Model Families

The tuned Decision Tree achieved the highest F1-score (0.7758) among all supervised models in these experiments. Without tuning, however, it was the worst-performing model (F1=0.7056), highlighting its high sensitivity to hyperparameters. SVM and Logistic Regression showed similar and stable performance (F1 approximately 0.7547 to 0.7577) across different experimental conditions, making them more reliable choices when tuning resources are limited. Random Forest was consistent but did not outperform the tuned DT in this case, though it required no tuning to achieve competitive results.

4.6 K-Means Clustering

The K-Means results suggest that job-seeking behaviour does not cluster naturally into two well-separated groups in the feature space. While the Silhouette Score of 0.53 indicates some internal cluster structure, the near-zero ARI (-0.018) and AMI (0.003) confirm these clusters do not align with the true labels. This is a common finding in behavioural datasets, where class boundaries are driven by complex feature interactions rather than geometric proximity.

4.7 Ensemble Models

Gradient Boosting came very close to matching the best individual model (F1=0.7757 vs 0.7758 for tuned DT), showing that well-configured boosting methods can be competitive with tuned individual classifiers. Stacking also performed reasonably well (F1=0.7619). The Voting Classifier with soft voting performed worst among ensemble methods (F1=0.7438), likely because averaging probabilities from models with different confidence calibrations can be counterproductive. Overall, ensemble methods did not substantially outperform the best individual models in these experiments.

4.8 Limitations and Potential Improvements

This work has some limitations worth considering. First of all, the high dimensionality after one-hot encoding (160 features) may affect distance-based methods like SVM and K-Means. Feature selection or dimensionality reduction could potentially improve results. Then, the class imbalance (75/25) was addressed through stratified splits and weighted metrics, but it is worth noting that weighted F1 can mask poor performance on the minority class. In this problem, class 1 (candidates actively seeking a job change) is arguably the more important class from a practical standpoint. Techniques such as using class_weight='balanced' in the models could further improve recall on this minority class. Additionally, the city feature generates a large number of columns after encoding, which may add noise without contributing much to the predictions.

5. Conclusion

This assignment implemented and evaluated a comprehensive set of machine learning models on the Job Change of Data Scientists dataset. The key findings are:

- Hyperparameter tuning has a large impact on tree-based models (DT F1 improved from 0.7056 to 0.7758 in these experiments) but minimal effect on linear models like Logistic Regression.
- Logistic Regression and SVM showed stable and consistent performance across different experimental conditions, making them reliable choices when tuning resources are limited.
- Stratification has minimal practical impact on this large dataset but remains recommended as best practice.
- In these experiments, the 70/30 split tended to offer a slightly better trade-off between training data and evaluation reliability.
- K-Means clustering confirmed that job-seeking behaviour does not have a clear two-cluster structure aligned with the true labels.
- Ensemble methods, especially Gradient Boosting, performed similarly to the best individual models but did not clearly outperform them in these experiments.

Overall, the tuned Decision Tree and Gradient Boosting achieved the highest F1-scores (0.7758 and 0.7757 respectively) in these experiments, while Logistic Regression offered a strong combination of performance and stability without requiring tuning.

6. Code Repository

All code for this assignment is available at:

https://github.com/marticasasn/ELEC3612-Assignment1

The repository is structured as follows:

- data/ — contains the dataset (aug_train.csv)
- notebooks/01_preprocessing.ipynb — EDA and preprocessing pipeline definition
- notebooks/02_supervised_models.ipynb — Decision Tree, Logistic Regression, Random Forest, and SVM with full robustness analysis
- notebooks/03_unsupervised_models.ipynb — K-Means clustering with elbow method, Silhouette, ARI, and AMI
- notebooks/04_ensemble_models.ipynb — Bagging, Boosting, Voting Classifier, and Stacking with comparison against base models
- report/ — final report PDF

Each notebook is self-contained and can be run independently. The preprocessing pipeline is defined consistently across all notebooks using the same build_pipeline() helper function, ensuring no data leakage across all experiments.